```
  }
});
```

But that will only load the data once, and Agtron wants to leave the application running and see the information about his servers update over time. A simple approach to getting regular data updates is to just ask for the data periodically.

How to run the listings

The remaining listings in this chapter are client-side and require a server to run. The downloadable code includes a server.coffee file that can be used to experiment with the listings. Run server.coffee from the command line and visit http://localhost:8080/ to start experimenting.

In the following listing, you see the techniques described in this section used in a first version of the dashboard program that repeatedly polls Agtron's servers to get the most recent data. Instead of displaying a graph, this first version simply displays the current numerical value as text.

Listing 11.2. The status updating script

This render function simply updates the page with the most recent values.

```
window.onload = ->
  status = document.querySelector '#status'

  render = (buffer) ->
    status.style.color = 'green'
    status.style.fontSize = '120px'
    status.innerHTML = buffer[buffer.length-1]

  nextCallbackId = do ->
    callbackId = 0
    -> callbackId = callbackId + 1

  nextCallbackName = ->
    "callback#{nextCallbackId()}"

  fetch = (src, callback) ->
    head = document.querySelector 'head'
    script = document.createElement 'script'
    ajaxCallbackName = nextCallbackName()
    window[ajaxCallbackName] = (data) ->
      callback data
    script.src = src + "?callback=#{ajaxCallbackName}"
    head.appendChild script

  seconds = (n) ->
    1000*n
```

The window.onload event is fired when the browser window has loaded, by which time it's ready to execute the program. For simplicity in this first version of the program, the entire thing is declared inside the function assigned to window.onload.

Some dynamic script insertion used to fetch data.